

Phase 2 - Model Checking using SPIN

Combining the Model with a Property File

Since SPIN requires the LTL formula and the model to reside in the same file, the generated property file must first be combined with `output.pml` before it can be checked. This is done using a simple file concatenation.

Linux / macOS

```
1 cat output.pml safety.pml > merged.pml
```

Windows (cmd)

```
1 copy /b output.pml+safety.pml merged.pml
```

The resulting `merged.pml` file is then passed to SPIN as usual. The same procedure is repeated for each property file (`liveness_N.pml`, `reachability.pml`, `invariant.pml`) with a different output name, or all can be verified against the same `merged.pml` name one at a time.

From this point on, only the `cat` form of the command is shown in each example below; on Windows, simply replace it with the equivalent `copy /b` form shown above.

Generated Verification Properties

Safety

```
1 ltl safety {  
2     [] (!divByZero)  
3 }
```

This is a safety property. It states that a "bad" condition – here, `divByZero` becoming true – must never hold at any point during execution. SPIN checks this by searching for any reachable state in which the condition holds; if none is found, the property is satisfied.

Liveness

```
1 ltl liveness_N {  
2     [] (main@inLoop_N -> <>main@exitLoop_N)  
3 }
```

This is a liveness property, generated once per loop (N is the loop index). It states that whenever execution enters a given loop, it must eventually exit it. SPIN checks this by searching for any execution path that enters the loop but never reaches its exit label; if none is found, the property is satisfied.

Reachability

```
1 ltl reachability {  
2     <>endReached  
3 }
```

This is a reachability property. It states that the program must eventually reach its final state, marked by `endReached` becoming true. SPIN checks this by searching for an execution path on which this state is reached; if at least one such path exists, the property is satisfied.

Invariant

```
1 ltl invariant {  
2     [] (condition)  
3 }
```

This is an invariant property. It states that a given condition must hold in every state throughout the entire execution. SPIN checks this by searching for any reachable state in which the condition is false; if none is found, the property is satisfied.

Verification Examples

Example 1 - Successful Verification

Hash Source

```
1 adad x = 10;
2 adad y = 2;
3 adad z = x / y;
4 ta (x > 0) {
5     x = x - 1;
6 }
```

Generated Promela (output.pml)

```
1 /* Generated Promela model from Hash */
2
3 bool divByZero = false;
4 bool outOfBound = false;
5 bool nullPointer = false;
6 bool noPermission = false;
7 bool endReached = false;
8 int x;
9 int y;
10 int z;
11
12 active proctype main() {
13     x = 10;
14     y = 2;
15     if
16     :: (y == 0) ->
17         divByZero = true;
18         goto endReached_label;
19     :: else ->
20         skip;
21     fi;
22     z = x/y;
23     loop_start_1:
24     do
25     :: (x>0) ->
26         inLoop_1:
27         x = x-1;
28         ;
29
30     :: else -> break
31     od;
32     exitLoop_1:
33     skip;
34
35     endReached = true;
36     endReached_label:
37     skip;
38 }
```

Safety Verification

```
1 cat output.pml safety.pml > merged.pml
2 spin -search -ltl safety merged.pml
```

Output

```
1 (Spin Version 6.5.2 -- 6 December 2019)
2 + Partial Order Reduction
3
4 Full statespace search for:
5 never claim + (safety)
6 assertion violations + (if within scope of claim)
7 cycle checks - (disabled by -DSAFETY)
8 invalid end states - (disabled by never claim)
9
10 State-vector 28 byte, depth reached 61, errors: 0
11 31 states, stored
12 1 states, matched
13 32 transitions (= stored+matched)
14 0 atomic steps
15 hash conflicts: 0 (resolved)
16
17 Stats on memory usage (in Megabytes):
18 0.002 equivalent memory usage for states (stored*(State-vector + overhead))
19 0.290 actual memory usage for states
20 128.000 memory used for hash table (-w24)
21 0.534 memory used for DFS stack (-m10000)
22 128.730 total actual memory usage
23
24
25 unreachable in proctype main
26 merged.pml:17, state 4, "divByZero = 1"
27 (1 of 21 states)
28 unreachable in claim safety
29 _spin_nvr.tmp:8, state 10, "-end-"
30 (1 of 10 states)
```

Explanation

SPIN verifies the generated model together with the safety property by exploring all reachable execution states. During the verification it checks whether the variable `divByZero` can ever become true. Since the divisor is initialized to a non-zero value, the error condition is never reached. The verification completes without reporting any violations, confirming that the safety property holds for all possible executions.

Liveness Verification

```
1 cat output.pml liveness.pml > merged.pml
2 spin -search -ltl liveness_1 merged.pml
```

Output

```
1 (Spin Version 6.5.2 -- 6 December 2019)
2 + Partial Order Reduction
3
4 Full statespace search for:
5 never claim + (liveness_1)
6 assertion violations + (if within scope of claim)
7 acceptance cycles + (fairness disabled)
8 invalid end states - (disabled by never claim)
9
10 State-vector 28 byte, depth reached 61, errors: 0
11 51 states, stored (71 visited)
12 28 states, matched
13 99 transitions (= visited+matched)
14 0 atomic steps
15 hash conflicts: 0 (resolved)
16
17 Stats on memory usage (in Megabytes):
18 0.003 equivalent memory usage for states (stored*(State-vector + overhead))
19 0.289 actual memory usage for states
20 128.000 memory used for hash table (-w24)
21 0.534 memory used for DFS stack (-m10000)
22 128.730 total actual memory usage
23
24
25 unreachable in proctype main
26 merged.pml:17, state 4, "divByZero = 1"
27 (1 of 21 states)
28 unreachable in claim liveness_1
29 _spin_nvr.tmp:10, state 13, "-end-"
30 (1 of 13 states)
```

Explanation

For this verification, SPIN translates the LTL formula into an internal *never claim* and checks it against the generated model. It explores the state space while searching for an execution that enters the loop but never reaches its exit. Because the value of `x` decreases on every iteration, the loop eventually terminates. No violating execution is found, so the liveness property is successfully satisfied.

Reachability Verification

```
1 cat output.pml reachability.pml > merged.pml
2 spin -search -ltl reachability merged.pml
```

Output

```
1 (Spin Version 6.5.2 -- 6 December 2019)
2 + Partial Order Reduction
3
4 Full statespace search for:
5 never claim + (reachability)
6 assertion violations + (if within scope of claim)
7 acceptance cycles + (fairness disabled)
8 invalid end states - (disabled by never claim)
9
10 State-vector 28 byte, depth reached 56, errors: 0
11 29 states, stored (58 visited)
12 27 states, matched
13 85 transitions (= visited+matched)
14 0 atomic steps
15 hash conflicts: 0 (resolved)
16
17 Stats on memory usage (in Megabytes):
18 0.002 equivalent memory usage for states (stored*(State-vector + overhead))
19 0.290 actual memory usage for states
20 128.000 memory used for hash table (-w24)
21 0.534 memory used for DFS stack (-m10000)
22 128.730 total actual memory usage
23
24
25 unreachable in proctype main
26 merged.pml:17, state 4, "divByZero = 1"
27 merged.pml:38, state 21, "-end-"
28 (2 of 21 states)
29 unreachable in claim reachability
30 _spin_nvr.tmp:6, state 6, "-end-"
31 (1 of 6 states)
```

Explanation

SPIN explores the state space to determine whether the program can reach its designated final state. The verification confirms that there exists an execution path leading to **endReached**. No property violation is reported during the search, indicating that the program can always terminate normally after completing its execution.

Deadlock Verification

Deadlock checking does not depend on any specific LTL property, so it is run directly on `output.pml` without merging any property file.

```
1 spin -search -deadlock output.pml
```

Output

```
1 proctype main
2 state 1 -(tr 3)-> state 2 [id 0 tp 2] [----G] output.pml:13 => x = 10
3 state 2 -(tr 4)-> state 8 [id 1 tp 2] [----G] output.pml:14 => y = 2
4 state 8 -(tr 5)-> state 4 [id 2 tp 2] [----G] output.pml:16 => ((y==0))
5 state 8 -(tr 2)-> state 7 [id 5 tp 2] [----G] output.pml:16 => else
6 state 4 -(tr 6)-> state 20 [id 3 tp 2] [----G] output.pml:17 => divByZero = 1
7 state 20 -(tr 1)-> state 21 [id 19 tp 2] [--e-L] output.pml:37 => (1)
8 state 21 -(tr 11)-> state 0 [id 20 tp 3500] [--e-L] output.pml:38 => -end-
9 state 7 -(tr 1)-> state 10 [id 6 tp 2] [----L] output.pml:20 => (1)
10 state 10 -(tr 7)-> state 15 [id 9 tp 2] [----G] output.pml:22 => z = (x/y)
11 state 15 -(tr 8)-> state 12 [id 10 tp 2] [----G] output.pml:25 => ((x>0))
12 state 15 -(tr 2)-> state 18 [id 12 tp 2] [----G] output.pml:25 => else
13 state 12 -(tr 9)-> state 15 [id 11 tp 2] [----G] output.pml:27 => x = (x-1)
14 state 18 -(tr 1)-> state 19 [id 17 tp 2] [----L] output.pml:33 => (1)
15 state 19 -(tr 10)-> state 20 [id 18 tp 2] [----G] output.pml:35 => endReached = 1
16
17 Transition Type: A=atomic; D=d_step; L=local; G=global
18 Source-State Labels: p=progress; e=end; a=accept;
19 Note: statement merging was used. Only the first
20 stmt executed in each merge sequence is shown
21 (use spin -a -o3 to disable statement merging)
```

Explanation

Deadlock checking is performed directly on the generated model without using an LTL property. SPIN explores every reachable state and verifies that there is always an enabled transition until the program terminates. Since no state is encountered in which execution becomes permanently blocked, the model is reported to be free of deadlocks.

Invariant Verification

```
1 cat output.pml invariant.pml > merged.pml
2 spin -search -ltl invariant merged.pml
```

Output

```
1 (Spin Version 6.5.2 -- 6 December 2019)
2 + Partial Order Reduction
3
4 Full statespace search for:
5 never claim + (invariant)
6 assertion violations + (if within scope of claim)
7 cycle checks - (disabled by -DSAFETY)
8 invalid end states - (disabled by never claim)
9
10 State-vector 28 byte, depth reached 61, errors: 0
11 31 states, stored
12 1 states, matched
13 32 transitions (= stored+matched)
14 0 atomic steps
15 hash conflicts: 0 (resolved)
16
17 Stats on memory usage (in Megabytes):
18 0.002 equivalent memory usage for states (stored*(State-vector + overhead))
19 0.290 actual memory usage for states
20 128.000 memory used for hash table (-w24)
21 0.534 memory used for DFS stack (-m10000)
22 128.730 total actual memory usage
23
24
25 unreachable in proctype main
26 merged.pml:17, state 4, "divByZero = 1"
27 (1 of 21 states)
28 unreachable in claim invariant
29 _spin_nvr.tmp:8, state 10, "-end-"
30 (1 of 10 states)
```

Explanation

SPIN checks that the specified invariant remains true throughout every reachable execution state. During the search, it attempts to find a state in which the condition is violated. Since the value of `x` decreases only to zero before the loop terminates, the invariant remains satisfied during the entire execution. Consequently, no violations are reported.

Example 2 - Safety Property Violation

Hash Source

```
1 adad x = 10;  
2 adad y = 0;  
3 adad z = x / y;
```

Generated Promela (output.pml)

```
1 /* Generated Promela model from Hash */  
2  
3 bool divByZero = false;  
4 bool outOfBound = false;  
5 bool nullPointer = false;  
6 bool noPermission = false;  
7 bool endReached = false;  
8 int x;  
9 int y;  
10 int z;  
11  
12 active proctype main() {  
13     x = 10;  
14     y = 0;  
15     if  
16     :: (y == 0) ->  
17         divByZero = true;  
18         goto endReached_label;  
19     :: else ->  
20         skip;  
21     fi;  
22     z = x/y;  
23  
24     endReached = true;  
25     endReached_label:  
26     skip;  
27 }
```

Verification Command

```
1 cat output.pml safety.pml > merged.pml
2 spin -search -ltl safety merged.pml
```

Verification Output

```
1 (Spin Version 6.5.2 -- 6 December 2019)
2 Warning: Search not completed
3 + Partial Order Reduction
4
5 Full statespace search for:
6 never claim + (safety)
7 assertion violations + (if within scope of claim)
8 cycle checks - (disabled by -DSAFETY)
9 invalid end states - (disabled by never claim)
10
11 State-vector 28 byte, depth reached 8, errors: 1
12 5 states, stored
13 0 states, matched
14 5 transitions (= stored+matched)
15 0 atomic steps
16 hash conflicts: 0 (resolved)
17
18 Stats on memory usage (in Megabytes):
19 0.000 equivalent memory usage for states (stored*(State-vector + overhead))
20 0.290 actual memory usage for states
21 128.000 memory used for hash table (-w24)
22 0.534 memory used for DFS stack (-m10000)
23 128.730 total actual memory usage
```

Explanation

The divisor is initialized to zero before the division operation is reached. As a result, the generated model sets the `divByZero` flag, which violates the safety property. SPIN detects this counterexample during the state-space exploration and reports a property violation. Since a violating execution has been found, the verification terminates with an error.

Example 3 - Liveness Property Violation

Hash Source

```
1 adad x = 5;
2 ta (x > 0) {
3     edame;
4 }
```

Generated Promela (output.pml)

```
1 /* Generated Promela model from Hash */
2
3 bool divByZero = false;
4 bool outOfBound = false;
5 bool nullPointer = false;
6 bool noPermission = false;
7 bool endReached = false;
8 int x;
9
10 active proctype main() {
11     x = 5;
12     loop_start_1:
13     do
14         :: (x>0) ->
15         inLoop_1:
16         goto loop_start_1;
17
18         :: else -> break
19     od;
20     exitLoop_1:
21     skip;
22
23     endReached = true;
24     endReached_label:
25     skip;
26 }
```

Verification Command

```
1 cat output.pml liveness.pml > merged.pml
2 spin -search -ltl liveness_1 merged.pml
```

Verification Output

```
1 (Spin Version 6.5.2 -- 6 December 2019)
2 Warning: Search not completed
3 + Partial Order Reduction
4
5 Full statespace search for:
6 never claim + (liveness_1)
7 assertion violations + (if within scope of claim)
8 acceptance cycles + (fairness disabled)
9 invalid end states - (disabled by never claim)
10
11 State-vector 28 byte, depth reached 9, errors: 1
12 5 states, stored
13 0 states, matched
14 5 transitions (= stored+matched)
15 0 atomic steps
16 hash conflicts: 0 (resolved)
17
18 Stats on memory usage (in Megabytes):
19 0.000 equivalent memory usage for states (stored*(State-vector + overhead))
20 0.290 actual memory usage for states
21 128.000 memory used for hash table (-w24)
22 0.534 memory used for DFS stack (-m10000)
23 128.730 total actual memory usage
```

Explanation

During verification, SPIN checks whether every execution that enters the loop can eventually reach its exit. The generated model repeatedly jumps back to the beginning of the loop without modifying the loop condition. This creates an infinite execution cycle, allowing SPIN to find a counterexample that violates the liveness property. Therefore, the verification reports an error.

Example 4 - Reachability Property Violation

Hash Source

```
1 adad x;  
2 ta (dorost) {}
```

Generated Promela (output.pml)

```
1 /* Generated Promela model from Hash */  
2  
3 bool divByZero = false;  
4 bool outOfBound = false;  
5 bool nullPointer = false;  
6 bool noPermission = false;  
7 bool endReached = false;  
8 int x;  
9  
10 active proctype main() {  
11     loop_start_1:  
12     do  
13     :: (true) ->  
14         inLoop_1:  
15         skip;  
16  
17     :: else -> break  
18     od;  
19     exitLoop_1:  
20     skip;  
21  
22     endReached = true;  
23     endReached_label:  
24     skip;  
25 }
```

Verification Command

```
1 cat output.pml reachability.pml > merged.pml
2 spin -search -ltl reachability merged.pml
```

Verification Output

```
1 (Spin Version 6.5.2 -- 6 December 2019)
2 Warning: Search not completed
3 + Partial Order Reduction
4
5 Full statespace search for:
6 never claim + (reachability)
7 assertion violations + (if within scope of claim)
8 acceptance cycles + (fairness disabled)
9 invalid end states - (disabled by never claim)
10
11 State-vector 20 byte, depth reached 3, errors: 1
12 2 states, stored
13 0 states, matched
14 2 transitions (= stored+matched)
15 0 atomic steps
16 hash conflicts: 0 (resolved)
17
18 Stats on memory usage (in Megabytes):
19 0.000 equivalent memory usage for states (stored*(State-vector + overhead))
20 0.291 actual memory usage for states
21 128.000 memory used for hash table (-w24)
22 0.534 memory used for DFS stack (-m10000)
23 128.730 total actual memory usage
```

Explanation

The loop condition is always true, preventing the execution from leaving the loop and reaching the end of the program. SPIN explores the reachable states and determines that no execution path can satisfy the reachability property. A counterexample is therefore produced, indicating that the final state is unreachable.

Example 5 - Deadlock Checking

Hash Source

```
1  adad x = 0;  
2  x = x + 1;
```

Generated Promela (output.pml)

```
1  /* Generated Promela model from Hash */  
2  
3  bool divByZero = false;  
4  bool outOfBound = false;  
5  bool nullPointer = false;  
6  bool noPermission = false;  
7  bool endReached = false;  
8  int x;  
9  
10 active proctype main() {  
11     x = 0;  
12     x = x+1;  
13  
14     endReached = true;  
15     endReached_label:  
16     skip;  
17 }
```

No property file is required for deadlock checking; `output.pml` is verified directly.

Verification Command

```
1 spin -search -deadlock output.pml
```

Verification Output

```
1 proctype main
2 state 1 -(tr 3)-> state 2 [id 0 tp 2] [----G] output.pml:11 => x = 0
3 state 2 -(tr 4)-> state 3 [id 1 tp 2] [----G] output.pml:12 => x = (x+1)
4 state 3 -(tr 5)-> state 4 [id 2 tp 2] [----G] output.pml:15 => endReached = 1
5 state 4 -(tr 1)-> state 5 [id 3 tp 2] [--e-L] output.pml:17 => (1)
6 state 5 -(tr 6)-> state 0 [id 4 tp 3500] [--e-L] output.pml:18 => -end-
7
8 Transition Type: A=atomic; D=d_step; L=local; G=global
9 Source-State Labels: p=progress; e=end; a=accept;
10 Note: statement merging was used. Only the first
11 stmt executed in each merge sequence is shown
12 (use spin -a -o3 to disable statement merging)
```

Explanation

SPIN analyzes the generated state space to determine whether execution can become permanently blocked. Every reachable state has at least one enabled transition until the program reaches its normal termination. Since no blocked states are found during the exploration, SPIN reports that the generated model is free of deadlocks.

Example 6 - Invariant Violation

Hash Source

```
1 adad x = 2;
2
3 ta (x >= 0) {
4     x = x - 1;
5 }
```

Generated Promela (output.pml)

```
1 /* Generated Promela model from Hash */
2
3 bool divByZero = false;
4 bool outOfBound = false;
5 bool nullPointer = false;
6 bool noPermission = false;
7 bool endReached = false;
8 int x;
9
10 active proctype main() {
11     x = 2;
12     loop_start_1:
13     do
14         :: (x>=0) ->
15         inLoop_1:
16         x = x-1;
17         ;
18
19         :: else -> break
20     od;
21     exitLoop_1:
22     skip;
23
24     endReached = true;
25     endReached_label:
26     skip;
27 }
```

Verification Command

```
1 cat output.pml invariant.pml > merged.pml
2 spin -search -ltl invariant merged.pml
```

Verification Output

```
1 (Spin Version 6.5.2 -- 6 December 2019)
2 Warning: Search not completed
3 + Partial Order Reduction
4
5 Full statespace search for:
6 never claim + (invariant)
7 assertion violations + (if within scope of claim)
8 cycle checks - (disabled by -DSAFETY)
9 invalid end states - (disabled by never claim)
10
11 State-vector 28 byte, depth reached 14, errors: 1
12 8 states, stored
13 0 states, matched
14 8 transitions (= stored+matched)
15 0 atomic steps
16 hash conflicts: 0 (resolved)
17
18 Stats on memory usage (in Megabytes):
19 0.000 equivalent memory usage for states (stored*(State-vector + overhead))
20 0.290 actual memory usage for states
21 128.000 memory used for hash table (-w24)
22 0.534 memory used for DFS stack (-m10000)
23 128.730 total actual memory usage
```

Explanation

The invariant requires the variable x to remain non-negative during the entire execution. However, the loop continues while $x \geq 0$, causing x to eventually become negative after the final iteration. SPIN identifies the reachable state that violates the invariant and reports the corresponding property violation. Therefore, the verification fails.